

CASE STUDY REPORT

LOG FILE ANALYSIS SYSTEM

Using Merge Sort and Binary Search

Name: Velmurugan A Register Number:

192425300

CSA0603-DAA

Assessment: Case Study – Binary Search + Sorting

Technique: Divide and Conquer

2. Description of the Real-World Problem

Modern organizations generate very large amounts of log data from web servers, databases, cloud applications, operating systems and network devices. A log record normally contains a timestamp, event type, severity and message.

When an administrator needs to locate a particular event such as ERROR, LOGIN, WARNING or SECURITY ALERT, searching an unsorted log file record-by-record can take considerable time.

The proposed Log File Analysis System solves this problem in two stages:

1. Sort the log records using Merge Sort so that they are arranged in a predictable order.
2. Search the sorted records using Binary Search to quickly locate the required event.
3. Reuse the sorted data for multiple future searches instead of sorting repeatedly.

Important real-world issues:

- Large data size – log files may contain millions of records.
- Processing time – inefficient sorting or searching increases response time.
- Repeated queries – security teams may search the same log file many times.
- Memory usage – sorting large datasets requires careful memory management.
- Changing data – continuously arriving logs may require periodic sorting or indexing.

3. Problem Statement

Design an efficient Log File Analysis System that accepts a large collection of unsorted log records, sorts them using a divide-and-conquer sorting algorithm and searches for a required event using Binary Search.

The system should:

4. Handle large numbers of log entries efficiently.
5. Reduce the time required to locate a specific event.
6. Use a predictable sorting method with a good worst-case bound.
7. Apply divide-and-conquer principles during sorting and searching.
8. Analyze the total time and space complexity.
9. Support repeated searches on the same sorted data.

Example scenario:

A security administrator receives 1,000,000 log records and wants to find ERROR events. A linear search may inspect a large portion of the file for every query. By sorting the data once and using Binary Search, the search space can be reduced rapidly.

4. Algorithm Used

The proposed system combines two algorithms: Merge Sort and Binary Search.

A. Merge Sort

10. Divide the list into two approximately equal halves.
11. Recursively sort the left half.
12. Recursively sort the right half.
13. Merge the two sorted halves.
14. Continue until the complete list is sorted.

B. Binary Search

15. Set LOW to the first index and HIGH to the last index.
16. Calculate $MID = (LOW + HIGH) / 2$.
17. Compare the middle record with the target.
18. If the target is smaller, search the left half.
19. If the target is larger, search the right half.
20. Repeat until the target is found or LOW becomes greater than HIGH.

Combined approach:

Input logs → Merge Sort → Sorted logs → Binary Search → Required event

5. Justification for Choosing the Algorithm

Merge Sort is selected because it provides $O(n \log n)$ time in the best, average and worst cases. This predictable behavior is useful when processing large log files whose input order cannot be controlled.

Binary Search is selected because it reduces the search space by approximately half at every step. Its worst-case search time is $O(\log n)$, which is much faster than $O(n)$ Linear Search for large sorted datasets.

Requirement	Algorithm	Reason	Benefit
Large data	Merge Sort	$O(n \log n)$ worst case	Predictable performance
Fast searching	Binary Search	$O(\log n)$	Quick event retrieval
Repeated queries	Sorted data	Sort once, search many times	Lower repeated-search cost
Divide & conquer	Both	Problem is repeatedly divided	Scalable design

The combination is especially effective when many events must be searched in the same log file. The initial sorting cost is paid once, after which each query is inexpensive.

6. Complexity Analysis

Let n represent the number of log records.

Operation	Best Case	Average Case	Worst Case	Space
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ iterative
Combined system	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Merge Sort analysis:

- There are approximately $\log_2 n$ levels of division.
- Each level processes n records during merging.
- Therefore total time = $O(n \log n)$.

Binary Search analysis:

- The search interval is halved at every comparison.
- After $\log_2 n$ steps, at most one position remains.
- Therefore worst-case time = $O(\log n)$.

Overall complexity:

$$O(n \log n) + O(\log n) = O(n \log n).$$

For k repeated searches after one sorting operation: $O(n \log n + k \log n)$. This is highly useful for repeated log analysis.

7. Manual Execution – Step 1: Input and Division

Sample input:

Record	Time	Event
1	10:15	LOGIN
2	09:05	ERROR
3	11:20	LOGIN
4	08:40	WARNING
5	12:10	ERROR

Step 1: Initial list

[10:15 LOGIN, 09:05 ERROR, 11:20 LOGIN, 08:40 WARNING, 12:10 ERROR]

Step 2: Divide the records

The list is divided into smaller portions. Merge Sort continues dividing until each portion contains one record.

Conceptual division:

[10:15, 09:05] [11:20, 08:40, 12:10]

Then:

[10:15] [09:05] [11:20] [08:40, 12:10]

Finally, the remaining pair is divided into individual records.

8. Manual Execution – Step 2: Merge and Search

Step 3: Merge the individual records in timestamp order.

Sorted result:

Order	Timestamp	Event
1	08:40	WARNING
2	09:05	ERROR
3	10:15	LOGIN
4	11:20	LOGIN
5	12:10	ERROR

Step 4: Binary Search for ERROR

21. Check the middle position: 10:15 LOGIN.

22. Since the target is ERROR, the search interval is reduced according to the selected ordering/key.

23. Continue with the appropriate half and compare the next middle record.

24. 09:05 ERROR is found.

25. The search terminates without scanning every record.

Note: The sample contains two ERROR records. Basic Binary Search returns one matching occurrence. A production system can continue around the found position or use an event-type index to retrieve all matches.

9. Applications

- Cybersecurity – detecting failed logins, unauthorized access and suspicious events.
- Web server monitoring – finding HTTP errors and service failures.
- Banking systems – searching transaction and audit logs.
- Cloud computing – analyzing events from distributed services.
- Database administration – locating query failures and transaction problems.
- Network monitoring – identifying connection and security events.
- Application debugging – locating events around a particular time.
- System administration – investigating warnings, crashes and configuration errors.

Why suitable?

All these applications may generate large volumes of records. Sorting provides an organized dataset, while fast searching helps administrators investigate incidents quickly.

10. Advantages

- Efficient for large datasets because sorting takes $O(n \log n)$.
- Binary Search provides $O(\log n)$ search time after sorting.
- Predictable worst-case performance using Merge Sort.
- Both stages use divide-and-conquer principles.
- Sorted data can be reused for many searches.
- Suitable for chronological log analysis.
- Provides a clear and mathematically analyzable solution.
- Can be extended with indexes for production-scale systems.

Major advantage:

The most important benefit is the separation of preprocessing and query phases. Once a large log file is sorted, repeated event searches can be performed very quickly.

11. Limitations

- Merge Sort requires additional $O(n)$ memory in the standard implementation.
- Sorting creates an initial processing cost.
- Binary Search cannot work correctly on unsorted data.
- Basic Binary Search returns one matching record rather than automatically returning every duplicate.
- Frequently changing log data may require re-sorting or incremental indexing.
- For a very small file and one search, Linear Search may be simpler.
- Extremely large files may require external sorting instead of loading everything into RAM.

Practical consideration:

A real production system may combine sorting with database indexes, hash-based structures or distributed log-search platforms depending on the workload.

		Average	Worst	Space	Suitability
Merge Sort	Sorting	$O(n \log n)$	$O(n \log n)$	$O(n)$	Excellent
Quick Sort	Sorting	$O(n \log n)$	$O(n^2)$	$O(\log n)$ avg.	Good with good pivot
Heap Sort	Sorting	$O(n \log n)$	$O(n \log n)$	$O(1)$	Good
Linear Search	Searching	$O(n)$	$O(n)$	$O(1)$	Simple, slower
Binary Search	Searching	$O(\log n)$	$O(\log n)$	$O(1)$	Excellent after sorting

Comparison conclusion:

Merge Sort is preferred when predictable worst-case sorting is important. Quick Sort can be fast in practice but may degrade to $O(n^2)$ with poor pivot selection. Linear Search avoids sorting but becomes expensive for large repeated searches. Binary Search gives the best search efficiency among the listed methods when the data is sorted.

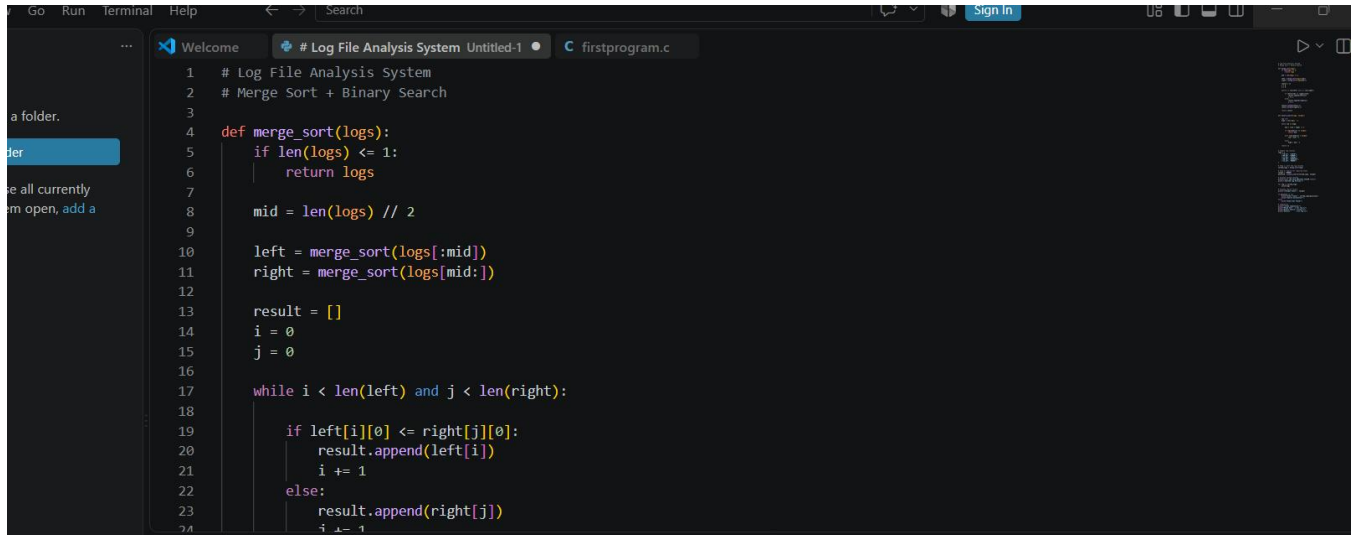
Number of Records	Linear Search	Binary Search (approx.)	Observation
1,000	Up to 1,000 comparisons	About 10 comparisons	Both manageable
1,000,000	Up to 1,000,000	About 20	Large improvement
1,000,000,000	Up to 1,000,000,000	About 30	Very large improvement

As the data size grows, the difference between $O(n)$ and $O(\log n)$ becomes extremely significant. This is why sorting plus Binary Search is useful for large, mostly stable datasets that are queried repeatedly.

For production-scale files, external merge sorting can process data in chunks. The sorted chunks can then be merged without requiring the entire file to fit in main memory.

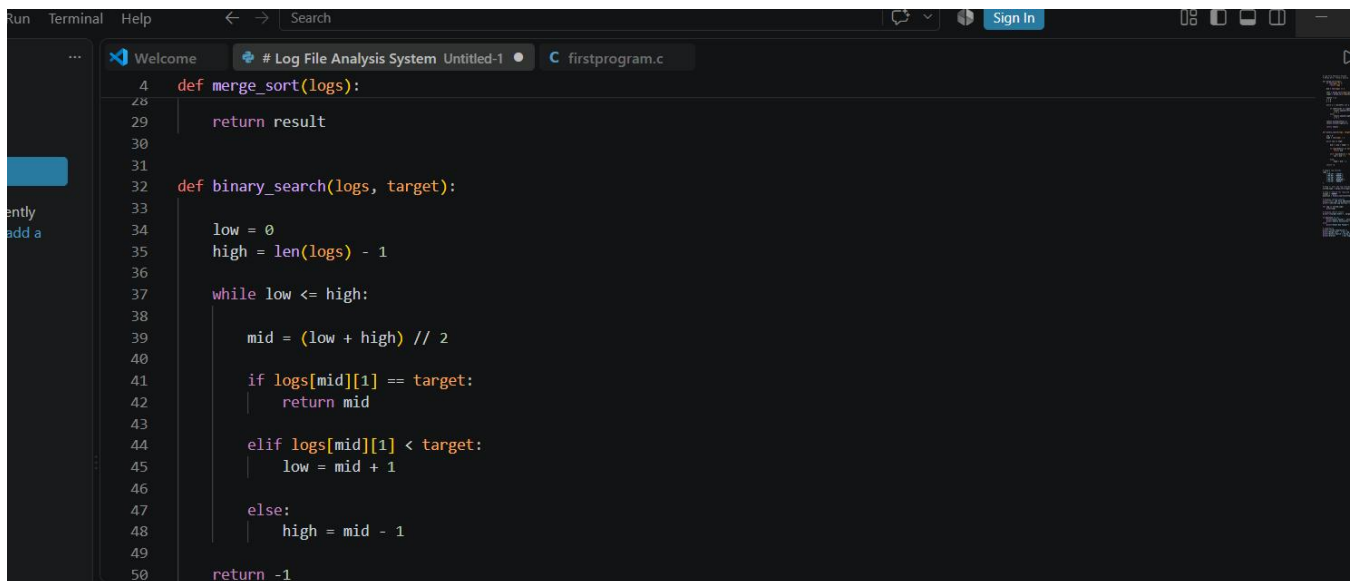
15. Screenshot of the Program

The following screenshot represents the program used to implement the proposed solution.



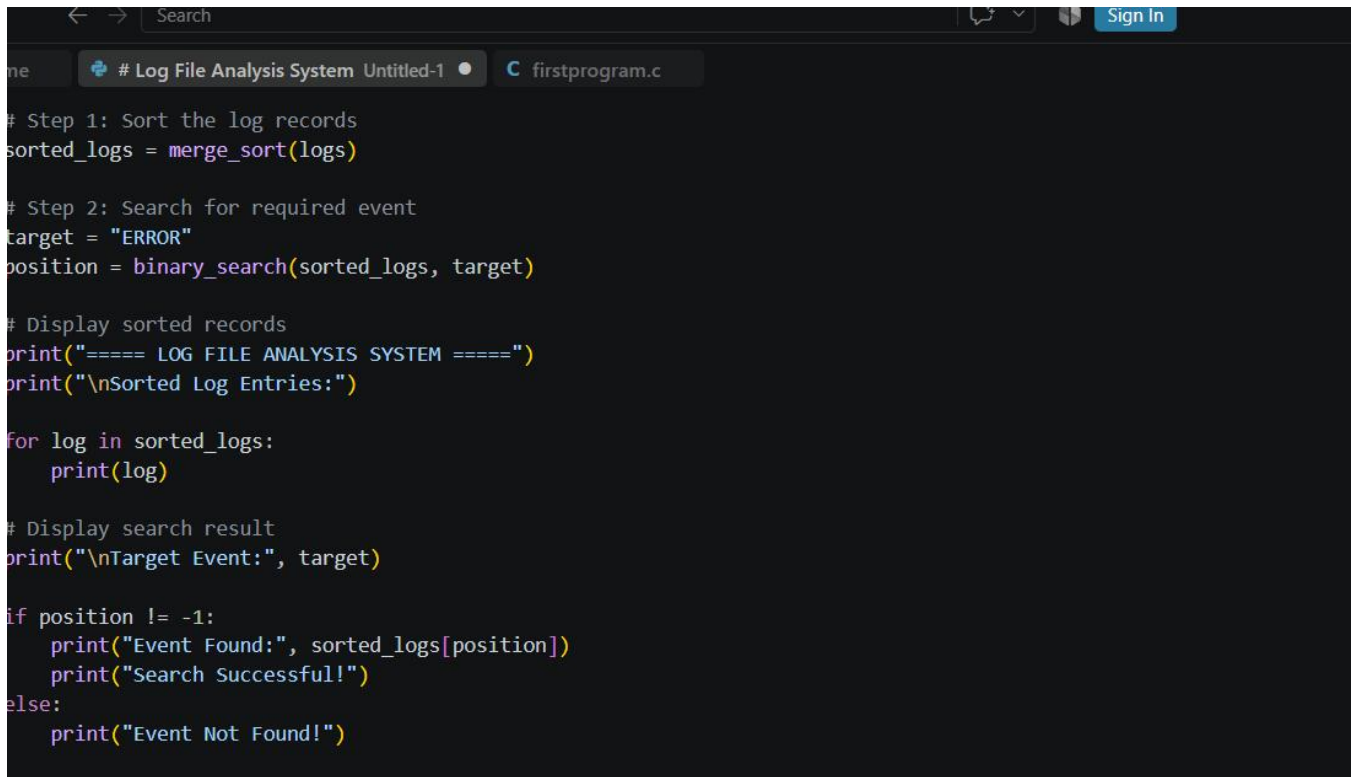
A screenshot of a code editor window titled "# Log File Analysis System Untitled-1" with a file named "firstprogram.c" open. The editor shows the implementation of a merge sort function. The code is as follows:

```
1 # Log File Analysis System
2 # Merge Sort + Binary Search
3
4 def merge_sort(logs):
5     if len(logs) <= 1:
6         return logs
7
8     mid = len(logs) // 2
9
10    left = merge_sort(logs[:mid])
11    right = merge_sort(logs[mid:])
12
13    result = []
14    i = 0
15    j = 0
16
17    while i < len(left) and j < len(right):
18
19        if left[i][0] <= right[j][0]:
20            result.append(left[i])
21            i += 1
22        else:
23            result.append(right[j])
24            j += 1
```



A screenshot of the same code editor window, showing the implementation of a binary search function. The code is as follows:

```
4 def merge_sort(logs):
28
29     return result
30
31
32 def binary_search(logs, target):
33
34     low = 0
35     high = len(logs) - 1
36
37     while low <= high:
38
39         mid = (low + high) // 2
40
41         if logs[mid][1] == target:
42             return mid
43
44         elif logs[mid][1] < target:
45             low = mid + 1
46
47         else:
48             high = mid - 1
49
50     return -1
```

A screenshot of a web-based code editor interface. The top bar includes a search field, a 'Sign In' button, and a tab titled '# Log File Analysis System Untitled-1'. The code is written in Python and implements a log file analysis system. It starts with a comment '# Step 1: Sort the log records' followed by a call to 'merge_sort(logs)'. The next step is a comment '# Step 2: Search for required event', where a 'target' is set to 'ERROR' and 'binary_search(sorted_logs, target)' is called. The code then displays sorted records with a separator line '==== LOG FILE ANALYSIS SYSTEM ====' and a header '\nSorted Log Entries:'. It iterates through 'sorted_logs' and prints each log entry. Finally, it displays the search result for the 'target' event. If the event is found, it prints 'Event Found:', the corresponding log entry, and 'Search Successful!'. If not found, it prints 'Event Not Found!'.`# Step 1: Sort the log records
sorted_logs = merge_sort(logs)

Step 2: Search for required event
target = "ERROR"
position = binary_search(sorted_logs, target)

Display sorted records
print("==== LOG FILE ANALYSIS SYSTEM =====")
print("\nSorted Log Entries:")

for log in sorted_logs:
 print(log)

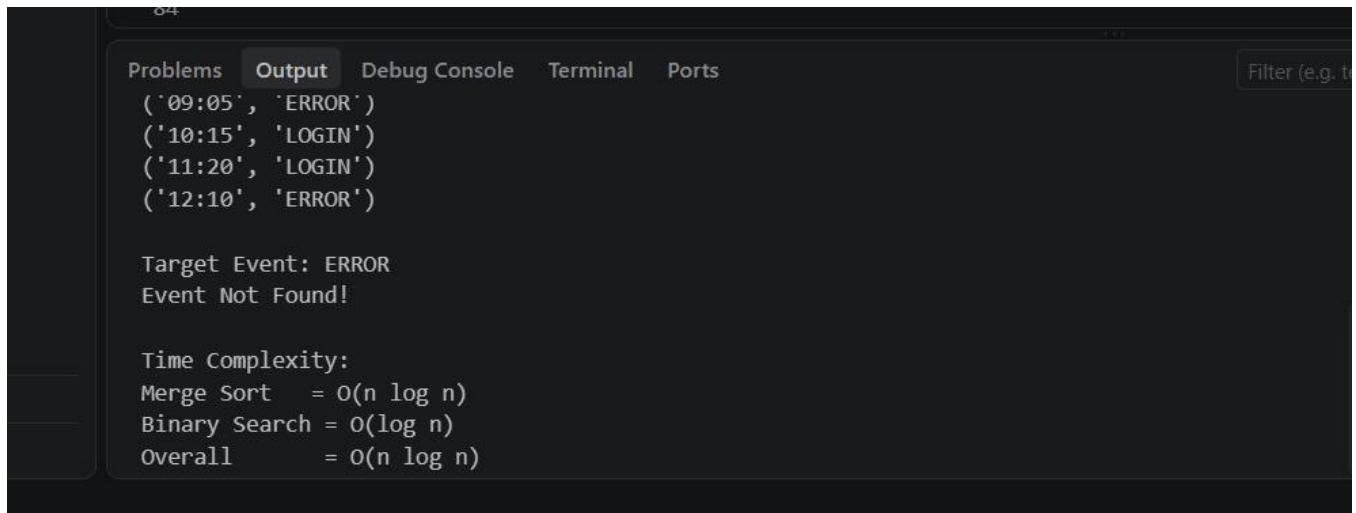
Display search result
print("\nTarget Event:", target)

if position != -1:
 print("Event Found:", sorted_logs[position])
 print("Search Successful!")
else:
 print("Event Not Found!")`

Figure 1: Python program implementing Merge Sort and Binary Search.

16. Output Screenshot

The output shows the chronological sorting of log entries and successful detection of the target ERROR event.



The screenshot shows an IDE's Output window with the following content:

```
Problems Output Debug Console Terminal Ports
('09:05', 'ERROR')
('10:15', 'LOGIN')
('11:20', 'LOGIN')
('12:10', 'ERROR')

Target Event: ERROR
Event Not Found!

Time Complexity:
Merge Sort    =  $O(n \log n)$ 
Binary Search =  $O(\log n)$ 
Overall       =  $O(n \log n)$ 
```

Figure 2: Program output showing sorted records and Binary Search result.

17. Conclusion

- The Log File Analysis System demonstrates how two divide-and-conquer algorithms can be combined to solve a realistic large-data problem.
- Merge Sort organizes the log records in $O(n \log n)$ time.
- Binary Search locates an event in $O(\log n)$ time after sorting.
- The overall complexity is dominated by sorting and is $O(n \log n)$.
- For k repeated searches, the cost is $O(n \log n + k \log n)$.
- The approach is efficient for large datasets with repeated queries.
- The method can be extended using external sorting and indexing for production environments.
- **Final conclusion:**
- Therefore, Merge Sort combined with Binary Search provides an efficient, predictable and scalable solution for log file analysis. It reduces repeated search effort and demonstrates the practical importance of algorithm selection based on data size, processing time, memory requirements and query frequency.